WEB APPLICATION BASICS

TryHackMe Learning & Practical Report

Prepared by	Belal Rabea Khalifa
Program	IqraSec Academy, WAPT Internship (Cohort 02), Phase 1
Task	1. Web Fundamentals
Platform	TryHackMe
Room	Web Application Basics (https://tryhackme.com/room/webapplicationbasics)
Focus	HTTP, URLs, requests and responses, status codes, headers, security headers
Completion	100% (all 11 tasks)
Date	October 2026

1. Objective

Understand how web applications work: HTTP/HTTPS, URLs, requests, responses, methods, headers and status codes. Then practice sending requests to an API.

2. Concept / Understanding

- **HTTP** (HyperText Transfer Protocol) is a stateless application layer protocol. It runs on top of TCP and is used to transfer resources like web app data.
- HTTP is the communication language between web browsers (clients like Chrome, Edge, Firefox) and web servers (like Apache, Nginx, IIS).
- **Stateless:** each request is independent. By default the server does not remember anything from the previous requests.
- **URL:** the location of a resource. It has a scheme (HTTP or HTTPS), a host, a path, and an optional query string that carries parameters.
- **HTTP messages:** the client sends a request, and the server sends back a response. The lines are separated by \r\n.
- **Request:** request line (method, path, version), headers, and an optional body. Methods: GET reads, POST sends data, PUT updates, DELETE removes, HEAD returns the headers only.
- **Response:** status line (version, status code, text), headers, and a body.
- **Status codes:** 2xx success (200 OK), 3xx redirect (301), 4xx client error (404 Not Found), 5xx server error (500).
- **Headers:** extra information. Request headers like Host, User-Agent, Content-Type, Cookie. Response headers like Server, Set-Cookie.
- **Security headers:** Content-Security-Policy (CSP) controls where scripts and resources can load from. HSTS forces the browser to use HTTPS. X-Content-Type-Options: nosniff stops the browser from guessing the content type.

Terms I noted:

Term	Meaning
client	the browser
version	HTTP/1.1
type	HTTP or HTTPS
\r\n	new line
body	the data
URL	the location
resource	the file

curl options I noted:

Option	Use
curl -v	verbose, shows the request and response details
curl -X	choose the method (GET, POST, PUT...)
curl -H	set a header (like User-Agent)
curl -I	request the page with the HEAD method (headers only)
curl -d	send data

3. Practical Work

I finished all 11 tasks of the room. In Task 10 (Practical Task: Making HTTP Requests), I used the room's HTTP request emulator and sent these 3 requests to the API:

#	Request	Goal
1	GET /api/users	get the user list
2	POST /api/user/2	update the country of Bob from UK to US
3	DELETE /api/user/1	delete the user

4. Tools Used

- TryHackMe room: Web Application Basics
- HTTP request emulator (Task 10)

5. Evidence

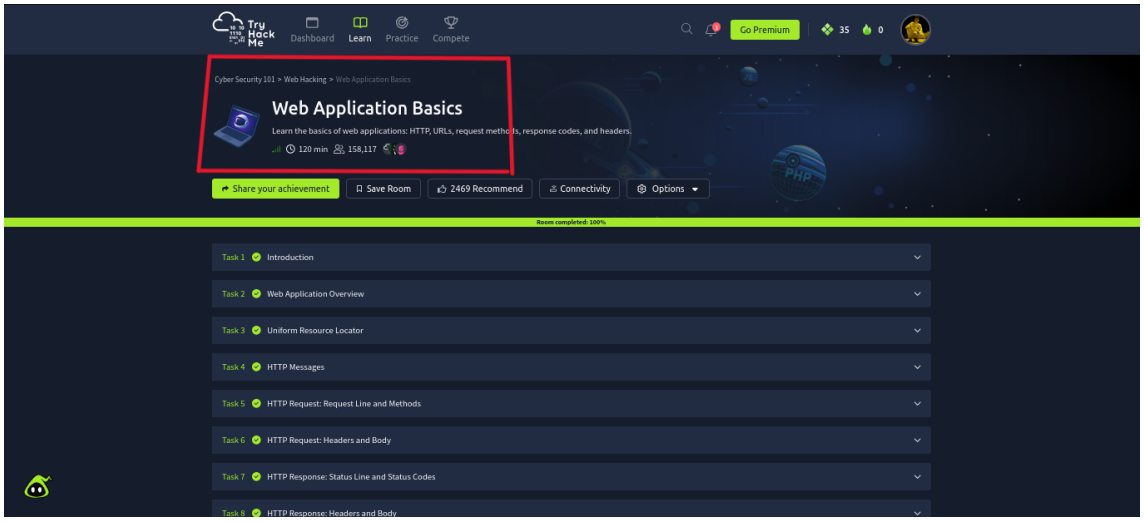


Figure 1: Web Application Basics room page, 100% completed.

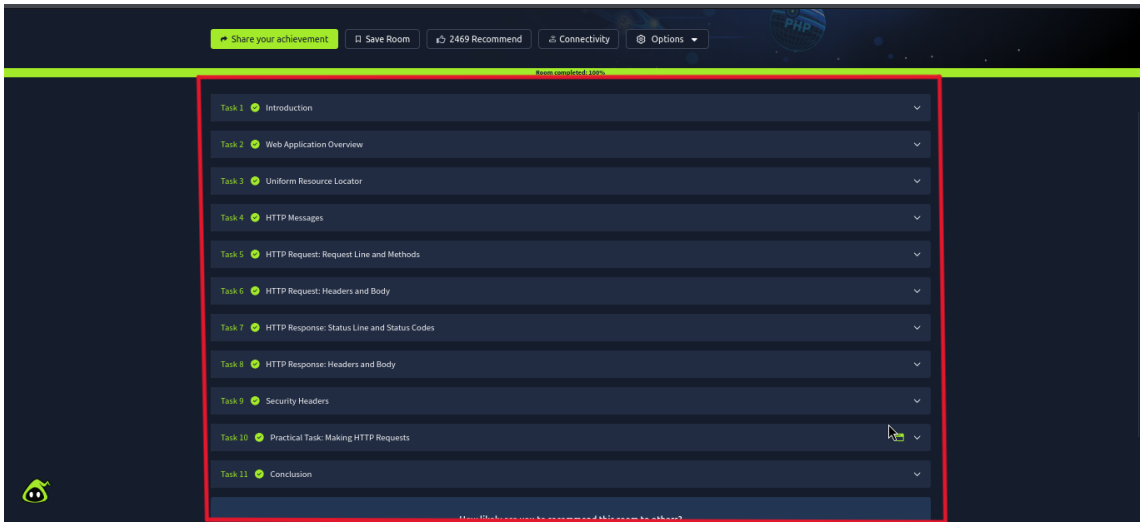


Figure 2: all 11 tasks of the room are completed.

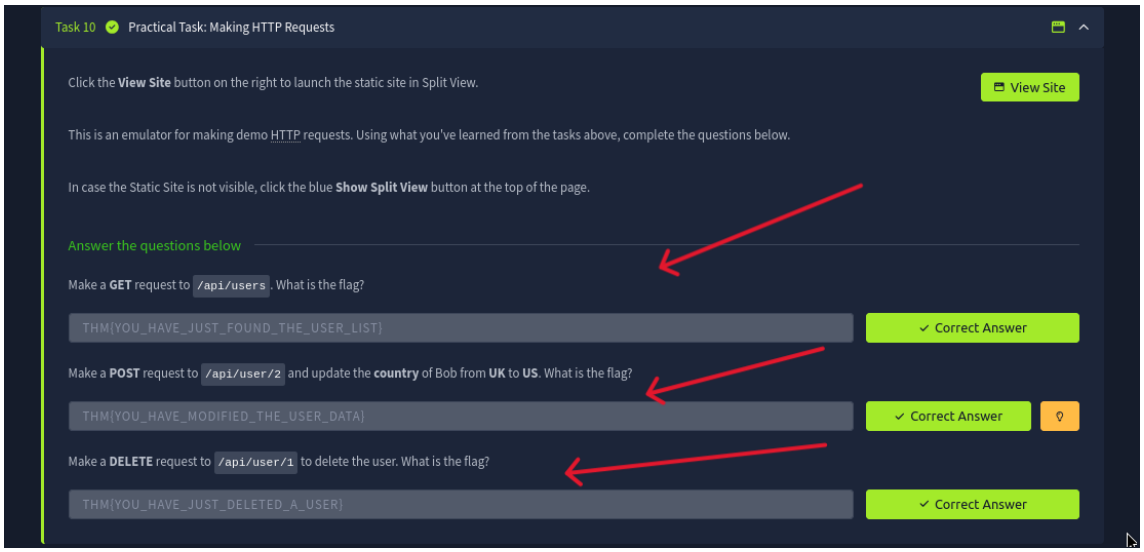


Figure 3: Task 10, the 3 requests (GET, POST, DELETE) with correct answers.

6. Result

Completed the room 100%. The GET, POST and DELETE requests to the API gave the correct flags.

7. Key Learnings

- Browsers talk to servers using HTTP/HTTPS, and HTTP runs on top of TCP.
- A request has a method, headers and sometimes a body. A response has a status code, headers and a body.
- Status codes tell what happened: 2xx success, 3xx redirect, 4xx client error, 5xx server error.
- Security headers like CSP, HSTS and nosniff protect the browser side of the application.